CISC 121 Assignment 1

Five Codility problems

GitHub repository:
https://github.com/K3shavC0des/cisc121-assignment-1

Problems and source statements

1. CyclicRotation
   https://app.codility.com/programmers/lessons/2-arrays/cyclic_rotation/

2. OddOccurrencesInArray
   https://app.codility.com/programmers/lessons/2-arrays/odd_occurrences_in_array/

3. FrogJmp
   https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/

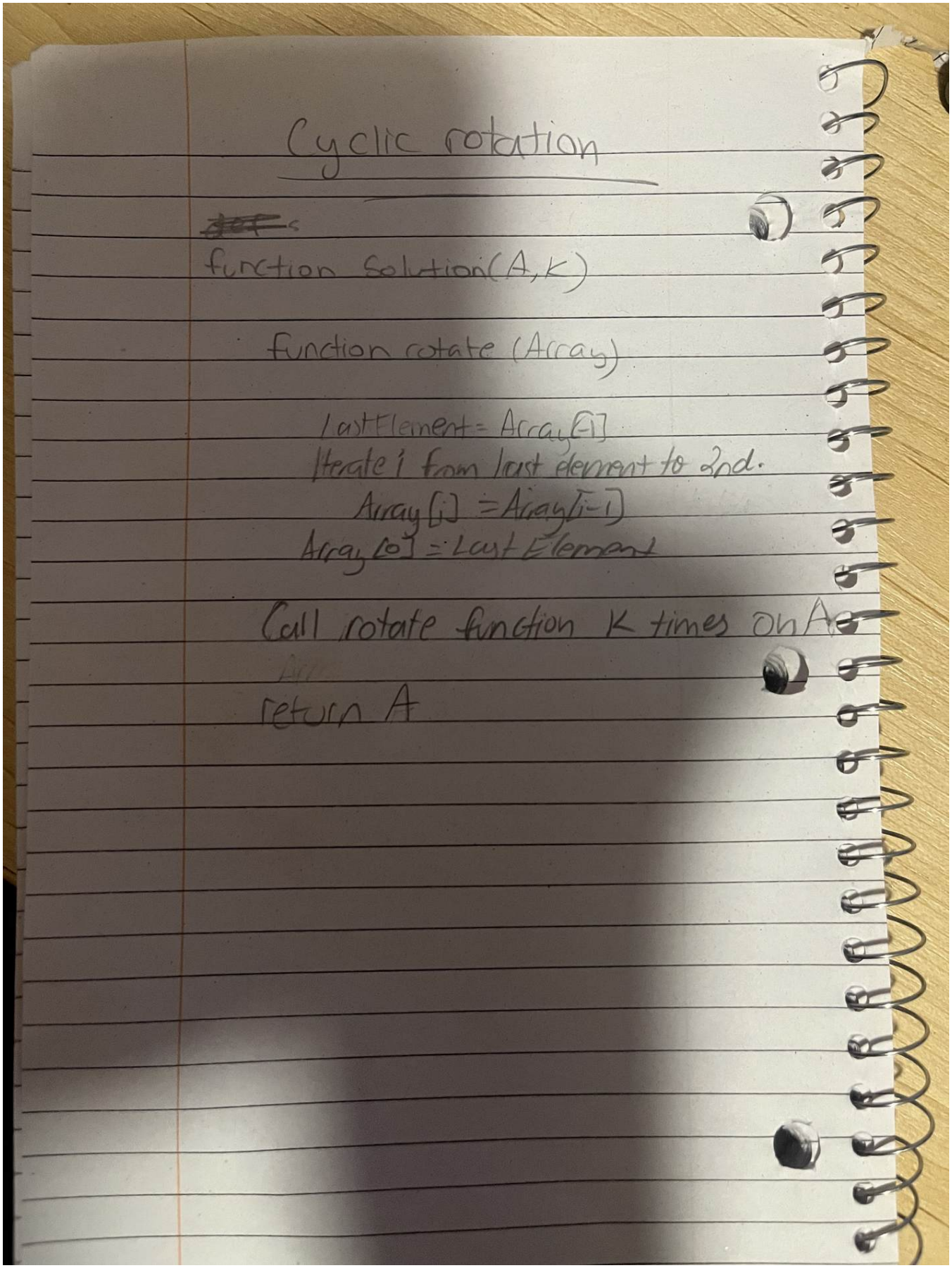
4. PermMissingElem
   https://app.codility.com/programmers/lessons/3-time_complexity/perm_missing_elem/

5. PermCheck
   https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/

Each problem has a handwritten pseudocode photograph and Python code on the following pages.

1. CyclicRotation

Handwritten pseudocode



1. CyclicRotation

Python code

Problem statement

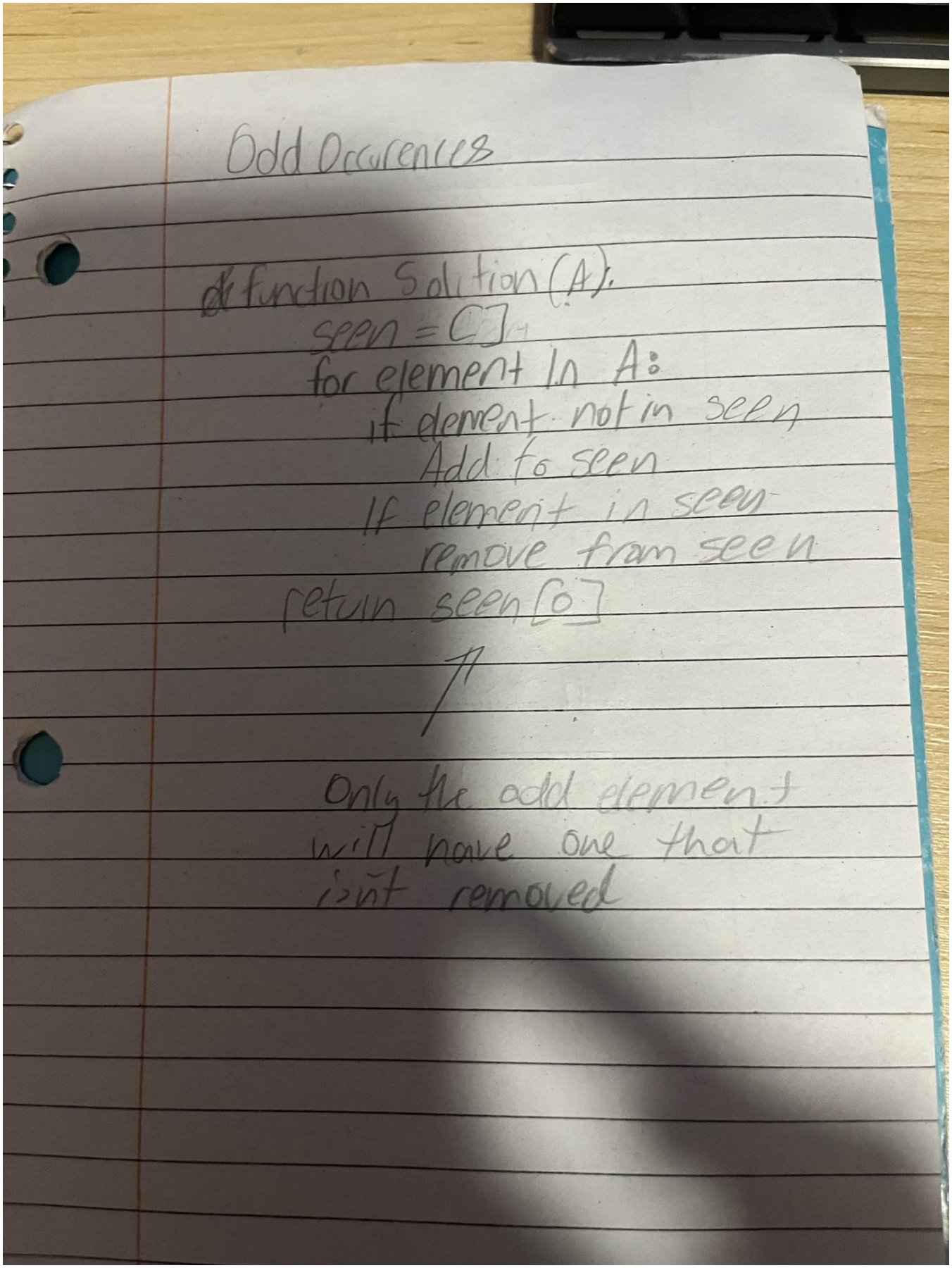
```
def solution(A, K):
    # An empty array stays empty after any number of rotations.
    if not A:
        return A

    # A full cycle of rotations leaves the array unchanged.
    K %= len(A)
    if K == 0:
        return A

    # Move the last K elements to the front.
    return A[-K:] + A[:-K]
```

2. OddOccurrencesInArray

Handwritten pseudocode



2. OddOccurrencesInArray

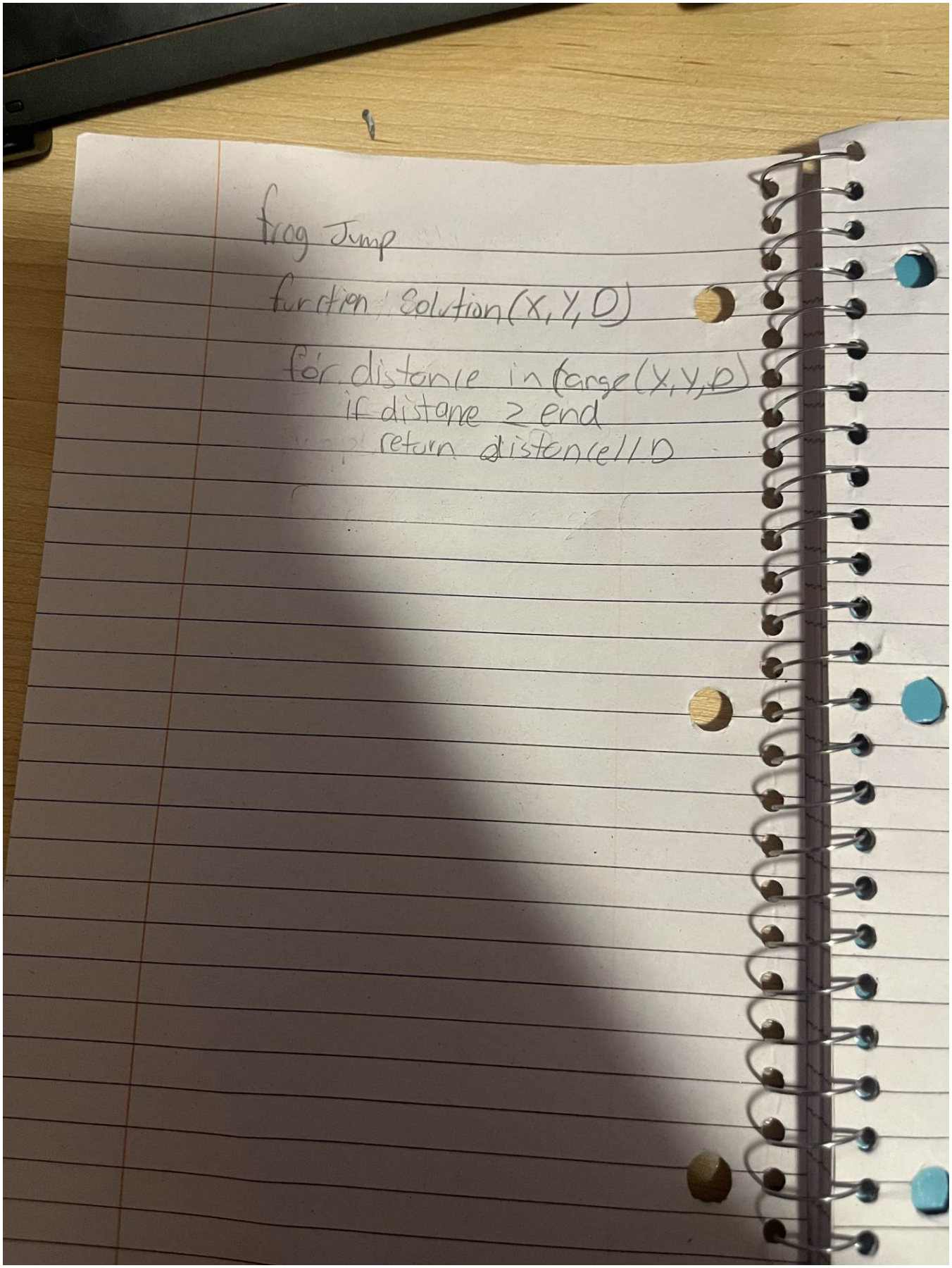
Python code

Problem statement

```
def solution(A):
    # Paired equal values cancel when XOR is applied.
    unpaired = 0
    for value in A:
        unpaired ^= value
    return unpaired
```

3. FrogJmp

Handwritten pseudocode



3. FrogJmp

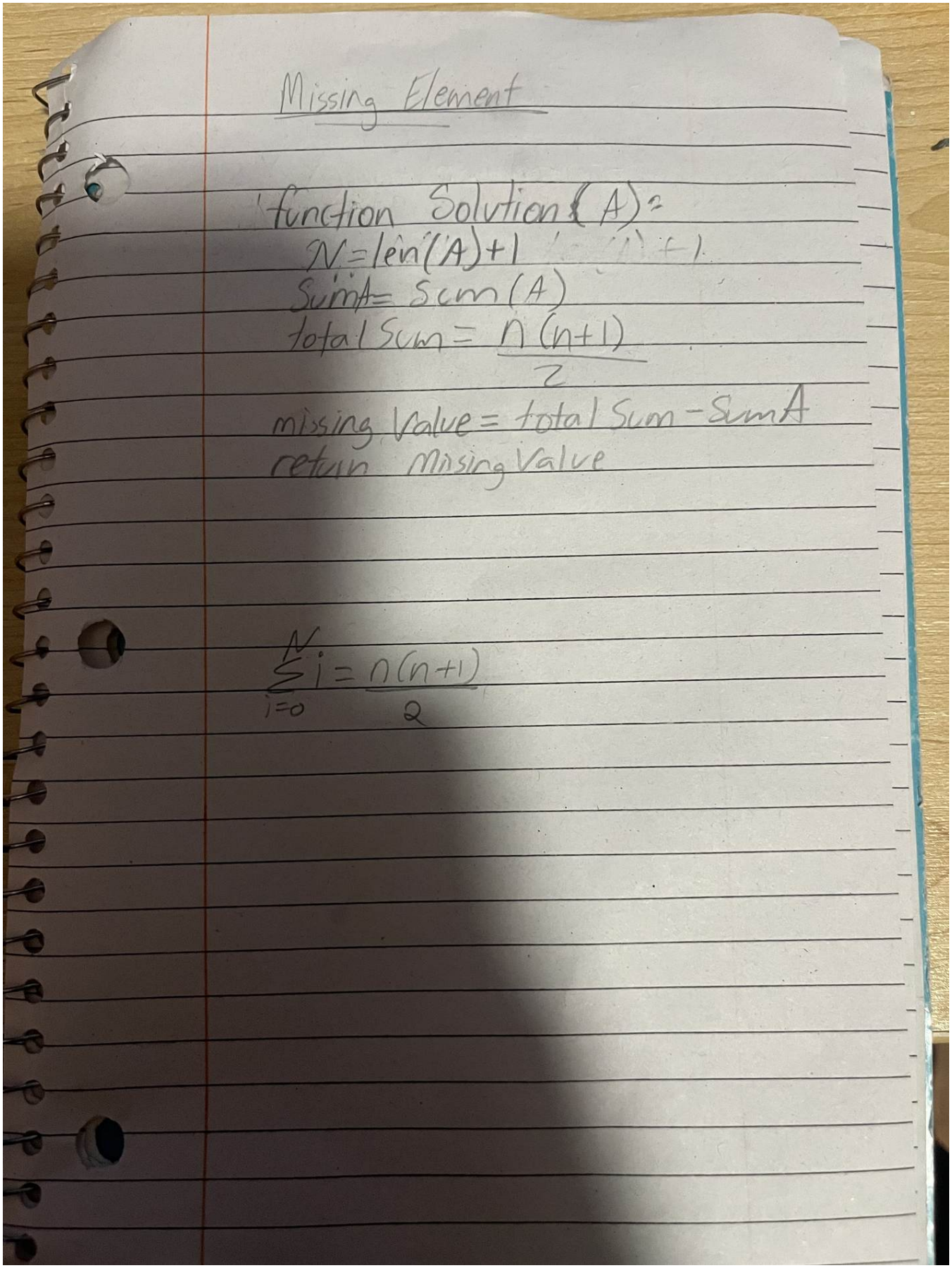
Python code

Problem statement

```
def solution(X, Y, D):
    # Round the remaining distance up to a whole number of jumps.
    distance = Y - X
    return (distance + D - 1) // D
```

4. PermMissingElem

Handwritten pseudocode



4. PermMissingElem

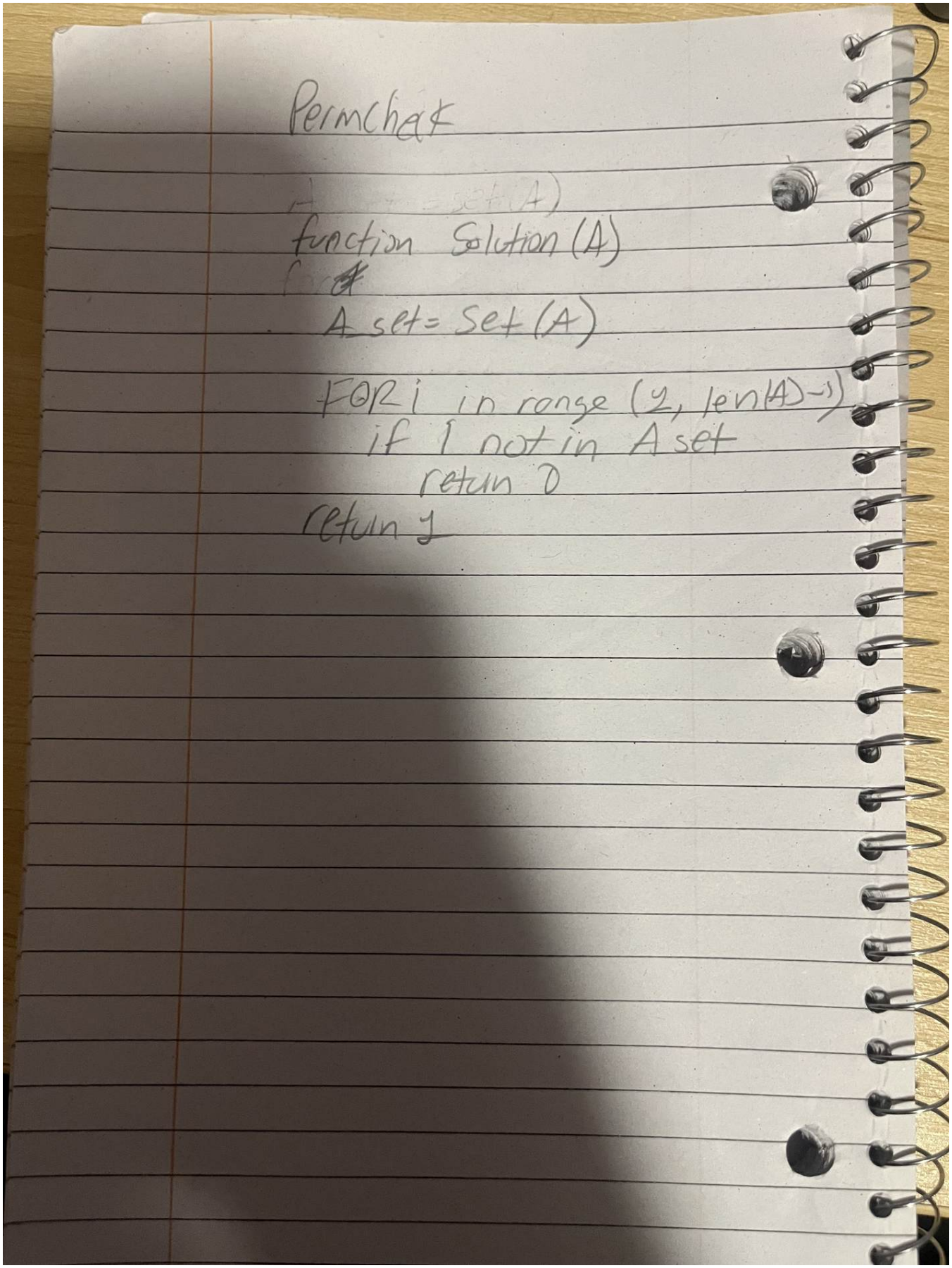
Python code

Problem statement

```
def solution(A):
    # The full permutation contains the integers from 1 through N.
    N = len(A) + 1
    expected_sum = N * (N + 1) // 2
    return expected_sum - sum(A)
```

5. PermCheck

Handwritten pseudocode



5. PermCheck

Python code

Problem statement

```
def solution(A):
    # A permutation of length N contains each integer from 1 through N.
    values = set(A)
    if len(values) != len(A):
        return 0

    for number in range(1, len(A) + 1):
        if number not in values:
            return 0
    return 1
```